

LLM-AGENT ARCHIVES

Сравнение двух сборок мультиагентной платформы

Глубокий разбор двух ZIP-архивов проекта llm-agent от 2025-09-25:
«github-ready» (с git-историей и dev-модулями) и «final» (чистый релиз).
Архитектура, метрики, зависимости, качество кода и 14 практических
рекомендаций по развитию кодовой базы.

695

файлов

90,5K

строк кода

37

MCP-серверов

35

агентов

ПОДГОТОВЛЕНО

Z.ai · Аналитический отчёт

ДАТА

01.10.2026

Содержание

Глава 1. Исполнительное резюме	3
Глава 2. Обзор проекта и архитектура	4
2.1. Стек технологий	4
2.2. Семислойная архитектура	4
2.3. Унифицированный принцип «всё — директория с манифестом»	5
Глава 3. Сравнение архивов (Diff)	5
3.1. Полный список различий	5
2.2. Семантика различий	6
Глава 4. Метрики кодовой базы	7
4.1. По типам файлов	7
4.2. По топ-каталогам	7
4.3. Подкаталоги src/	8
4.4. Крупнейшие файлы	9
Глава 5. Карта модулей	10
5.1. Ядро (src/)	10
5.2. Подсистемы данных	11
5.3. MCP-серверы (src/mcp_servers/)	11
5.4. Декларативный слой и тестирование	12
Глава 6. Зависимости и стек	13
6.1. Зависимости по категориям	13
6.2. Особенности и риски стека	14
Глава 7. Запуск и деплой	14
7.1. Точки входа и установщики	14
7.2. Docker Compose — инфраструктура	15
7.3. Systemd и nginx	15
7.4. Браузерное расширение	16
Глава 8. Качество кода и безопасность	16
8.1. Подробности по рискам	17
8.2. Архитектурные долги	17
Глава 9. Выводы и рекомендации	18

9.1. Дорожная карта развития 19

9.2. Заключительная оценка 19

Глава 1. Исполнительное резюме

Предметом анализа стали два ZIP-архива проекта **llm-agent** от 25 сентября 2026 года: *llm-agent-github-ready-2026-09-25 (1).zip* (≈3 МБ) и *llm-agent-final-2026-09-25-3 (2).zip* (≈1,3 МБ). Оба архива содержат одну и ту же мультиагентную LLM-платформу для локального веб-чата с моделью — систему, которая через декларативные MCP-серверы управляет файлами проекта, базами данных и 1С-конфигурациями. Платформа построена на FastAPI + uvicorn, использует PostgreSQL с расширением pgvector для векторной памяти, Apache AGE для property-graph, Kafka для CDC, Ollama как малую LLM для фонового обогащения и Redis для межинстансной координации.

Ключевое наблюдение сравнения: **архив «final» является строгим подмножеством «github-ready»** и получен из него удалением 16 файлов (без правки каких-либо общих). Удалены три группы содержимого: (1) каталог `.git/` с git-историей (один коммит, ≈5 МБ), (2) четыре JSON-профиля в `data/profiles/` (*default, development, production, 1c_development*) и (3) экспериментальные подсистемы `build` и `data` — вместе с декларациями агентов, декларациями MCP-серверов и их Python-реализациями в `src/mcp_servers/`. Все остальные 695 файлов остались байт-в-байт идентичны. Это указывает на то, что «github-ready» — это полный снимок репозитория (с историей и dev-модулями), а «final» — это чистый релизный дифакт без экспериментов.

Объём кодовой базы финальной сборки: **695 файлов, 90 576 строк кода**, из которых **67 741 строка** приходится на Python (408 файлов с расширением `.py`). Декларативный слой — 134 YAML-файла (4 895 строк) и 102 markdown-документа (5 719 строк). Браузерное расширение (Chromium/Firefox) добавляет 15 JavaScript-файлов (5 653 строки). В системе заявлено **37 MCP-серверов** и **35 декларативных агентов** в финальной сборке (в github-ready — 39 и 37 соответственно за счёт `build` и `data`).

695	67,7K	37 / 35
файлов в финальной сборке	строк Python-кода	MCP-серверов / агентов

Главные выводы. (1) Кодовая база архитектурно зрелая: декларативный реестр агентов и MCP-серверов на YAML с pydantic-валидацией, единая точка перехвата tool-calls в `src/mcp_manager.py`, LoopController с бюджетами и телеметрией, журнал действий (event sourcing) с откатом и реплеем, 5-слойная память (working, episodic, semantic, vector, procedural). (2) Качество кода в целом высокое: ноль вызовов `eval()`, ноль `shell=True` в `subprocess`, ноль `pickle.loads()`, ноль небезопасных `yaml.load()` (только `safe_load`), ноль `bare-except`. (3) Главная проблема maintainability — монолитный файл `src/main.py` объёмом **2 640 строк**, содержащий всю FastAPI-логику, WebSocket-обработчики и админ-API в одном модуле. (4) Финальная сборка лишилась runtime-профилей, которые система ожидает найти в `data/profiles/` — их необходимо восстановить либо документировать, что `first_run.py` их регенерирует.

Дальнейшие главы раскрывают эти выводы детально: глава 2 описывает архитектуру, глава 3 — посимвольное сравнение архивов, глава 4 — количественные метрики, глава 5 — карту модулей, глава 6 — стек зависимостей, глава 7 — запуск и деплой, глава 8 — результаты сканирования безопасности и

качества, глава 9 — 14 практических рекомендаций.

Глава 2. Обзор проекта и архитектура

Проект **llm-agent** — это локальный веб-чат с LLM, который через MCP-серверы (Model Context Protocol) управляет файлами проекта и базами данных. Пользователь общается с системой через браузерный интерфейс на `http://127.0.0.1:8000`, а система маршрутизирует запрос в подходящего агента, который вызывает инструменты MCP-сервера (чтение файлов, SQL-запросы, операции с 1C, Git, shell-команды и т. д.) и возвращает результат. Поддерживается полный цикл 1C-разработки: чтение метаданных, выполнение запросов, сборка конфигураций, прогон тестов.

2.1. Стек технологий

Стек полностью описан в `requirements.txt` и охватывает 12 категорий зависимостей: web-сервер (FastAPI + uvicorn), LLM-клиент (openai, httpx, aiohttp), MCP-протокол (mcp>=1.1.3,<2.0 — пиннинг критичен, потому что mcp 2.x удалил декораторы `@app.list_tools()`), валидация (pydantic 2.9+, pydantic-settings), базы данных (psycopg3, pgvector, pymysql, lancedb), векторная память (sentence-transformers с моделью BAAI/bge-m3 для русского языка), tree-sitter с 8 грамматиками (Python, JS/TS, Go, Rust, Java, C#, PHP, Ruby) для построения графа кода, Playwright для browser-MCP, faster-whisper для аудио, Redis и aiokafka для кластера и CDC, kubernetes-asyncio для управления Kubernetes-кластером, rich для CLI-вывода. Общий объём зависимостей — около 70 пакетов; для Python 3.13t/3.14t (free-threaded сборки) предусмотрен отдельный файл `requirements-freethreaded.txt` с усечённым набором (без lancedb, sentence-transformers, tree-sitter, duckdb, aiokafka, faster-whisper — у них нет t-колёс на PyPI).

2.2. Семислойная архитектура

Кодовая база организована в семь горизонтальных слоёв, каждый из которых независимо тестируется и конфигурируется через YAML-декларации. Архитектурный документ `docs/ARCHITECTURE-V2.md` прямо указывает, что проект уже имеет «сильный декларативный фундамент»: реестр агентов, MCP-серверов, capabilities и loop-спеков, единая точка tool-calls, LoopController с бюджетами и телеметрией, snapshot/A-B и журнал действий. Главная архитектурная претензия автора к самому себе — «роутинг без памяти диалога» (`route()` не видит историю) и «нет исполнителя плана» (после `route()` агенты выполняются последовательно на одном тексте без pipeline и re-plan). Предлагается Supervisor-цикл Plan→Execute→Observe→Re-plan.

Слой	Каталог	Назначение
L1 — Web	src/main.py, src/web/	FastAPI-сервер, WebSocket-стриминг, REST-API
L2 — Orchestrator	src/orchestrator.py	LLM-роутинг между агентами по <code>routing_hints</code>
L3 — Supervisor	src/supervisor/	DAG-планы: Plan→Execute→Observe→Re-plan
L4 — LoopController	src/loop/	Декларативные циклы reasoning/verification/retry с бюджетами

Слой	Каталог	Назначение
L5 — Agents	agents/, src/agents/, src/prompts/	37 декларативных агентов (YAML + prompt.md)
L6 — MCP servers	mcp_servers/, src/mcp_servers/	37 MCP-серверов: файлы, Git, БД, 1C, browser...
L7 — Capabilities	capabilities/	Vision, OCR, Whisper, Media, Storage, 1C-metadata

Таблица 2.1. Семислойная архитектура llm-agent

Дополнительно к семи слоям в проекте есть четыре инфраструктурных подсистемы: **src/db/** (PostgreSQL pool, vector_store на pgvector, hybrid_search с RRF, analytics, AGE store для Cypher, CDC Kafka publisher, pg_dump backup), **src/memory/** (5 слоёв памяти: working, episodic, semantic, vector, procedural; retriever, summarizer, embedders с fallback на hashing), **src/journal/** (event sourcing: recorder, replay, rollback, retention, schema, store — полный аудит tool-calls), **src/cluster/** (multi-instance через Redis bus, heartbeat, WS bridge для горизонтального масштабирования). Эти подсистемы включаются опционально через config/settings.yaml и могут работать в деградированном режиме (например, без PostgreSQL — на SQLite + LanceDB).

2.3. Унифицированный принцип «всё — директория с манифестом»

В проекте последовательно проведён принцип: каждый модуль — это каталог с YAML-манифестом. Агенты описаны в agents//agent.yaml (поля: id, schema_version, title, description, requires.python_packages/env_vars/paths, mcp_servers, depends_on.hard/soft, priority, routing_hints.keywords/negative_keywords/description_for_router, mode: read|write|destructive, dangerous_tools, prompt, user_template, runtime.max_steps/max_result_chars/timeout_seconds, ui.icon/color/category). MCP-серверы — в mcp_servers//server.yaml (id, command, args, requires, mode, tools[{name, danger: read|write|destructive}], process.startup_timeout_seconds/restart_policy/max_restarts/backoff_seconds). Capabilities — в capabilities/.yaml с operations[] и providers[]. Loops — в loops/*.yaml с trigger/budget/body[{kind: llm_call|tool_calls, params}]/on_success/on_failure/on_timeout/metrics. Features (журнал, ops, bridge, cluster, history, memory, notes) — в features//feature.yaml.

Глава 3. Сравнение архивов (Diff)

Сравнение выполнено через find по обоим архивам с последующим diff отсортированных списков файлов. Из анализа исключены каталоги .git/ (git-история), extension/dist/ (собранные артефакты расширения) и extension/icons/ (PNG-иконки). Результат однозначен: **все 695 файлов финальной сборки байт-в-байт идентичны файлам из github-ready** (проверено через MD5-хеши топ-уровневых файлов и точное сравнение содержимого выборочных файлов). Архив «final» не содержит ни одного файла, которого не было бы в «github-ready». Разница — только в 16 файлах, удалённых из github-ready при формировании final.

3.1. Полный список различий

Категория	Путь (только в github-ready)	Тип
Git-история	.git/ (каталог, ≈5 МБ, 1 коммит)	VCS
Профиль 1	data/profiles/default.json	Runtime profile
Профиль 2	data/profiles/development.json	Runtime profile
Профиль 3	data/profiles/production.json	Runtime profile
Профиль 4	data/profiles/1c_development.json	Runtime profile
Агент build	agents/build/agent.yaml	Agent YAML
Агент build	agents/build/prompt.md	Agent prompt
Агент build	agents/build/user.md	Agent template
Агент data	agents/data/agent.yaml	Agent YAML
Агент data	agents/data/prompt.md	Agent prompt
Агент data	agents/data/user.md	Agent template
MCP build	mcp_servers/build/server.yaml	MCP declaration
MCP data	mcp_servers/data/server.yaml	MCP declaration
MCP build	src/mcp_servers/build/__init__.py	Python impl
MCP build	src/mcp_servers/build/server.py	Python impl
MCP data	src/mcp_servers/data/__init__.py	Python impl
MCP data	src/mcp_servers/data/server.py	Python impl

Таблица 3.1. 16 файлов, присутствующих в github-ready, но отсутствующих в final

2.2. Семантика различий

- (1) **Git-история.** В архиве «github-ready» лежит полный каталог `.git/` объёмом 5 МБ — это инициализированный git-репозиторий с одним коммитом «LLM Agent — локальная агентная платформа (v2026-09-25)» на ветке `main`, без тегов. По сути — это снимок первого коммита в репозиторий, готовый к публикации на GitHub. В «final» этот каталог удалён — финальная сборка не предполагает, что пользователь будет видеть историю разработки или пушить изменения обратно.
- (2) **Runtime-профили.** Четыре JSON-файла в `data/profiles/` (*default*, *development*, *production*, *1c_development*) — это предустановленные конфигурационные профили системы: каждый профиль отключает определённые агенты и/или переопределяет параметры (например, в *production* отключены агенты `shell` и `git`; в *1c_development* отключены `mysql` и `postgres`, повышены приоритеты `file` и `openc`). Профили помечены `is_builtin: true` и имеют временные метки `created_at: 1790270986` (соответствует ~2026-09-25). Удаление этих файлов из финальной сборки — это, **вероятно, ошибка**: код системы ожидает, что профили лежат в `data/profiles/`, и без них первый запуск не найдёт встроенных пресетов (подробности — глава 9, рекомендация 2).
- (3) **Экспериментальные MCP-серверы build и data.** В «github-ready» присутствуют два экспериментальных агента и два MCP-сервера, которые в «final» удалены: *build* — сборка, зависимости

и Docker (16 инструментов: `detect_project`, `build_run`, `pkg_add`, `docker_build`, `docker_compose_up` и т. д., режим *destructive*) и *data* — управление данными (предположительно ETL-операции). Оба агента имеют `mode: destructive` и помечены как *dangerous_tools* — то есть это потенциально небезопасные операции, и их удаление из финальной сборки выглядит как сознательная санация перед релизом. В Python-исходниках `src/mcp_servers/build/` и `src/mcp_servers/data/` лежат `__init__.py` и `server.py` — это реальные имплементации MCP-серверов, удалённые из финальной сборки.

Глава 4. Метрики кодовой базы

Метрики собраны скриптом `scripts/analyze_loc.py`, который рекурсивно обходит каталоги, исключая `.git/`, `__pycache__`, `extension/dist/` и `extension/icons/`. Для каждого файла считается количество строк (по байт-разделителям `\n`), а затем данные агрегируются по расширению и по топ-уровневому каталогу. Все цифры в этой главе относятся к финальной сборке; `github-ready` больше на 16 файлов и ~1 164 строки Python-кода.

4.1. По типам файлов

Расширение	Файлов	Строк	Доля строк
.py	408	67 741	74,8%
.md	102	5 719	6,3%
.yaml	134	4 895	5,4%
.js	15	5 653	6,2%
.html	6	1 745	1,9%
.sql	7	1 603	1,8%
.css	3	1 374	1,5%
.bat	4	393	0,4%
.txt	5	380	0,4%
прочее	11	1 173	1,3%
ИТОГО	695	90 576	100%

Таблица 4.1. Распределение файлов по типам (финальная сборка)

Python доминирует (74,8% строк), что естественно для backend-проекта. Доля YAML (5,4%) и Markdown (6,3%) отражает декларативный подход: 134 YAML — это декларации агентов, MCP-серверов, *capabilities*, *loops*, *features* и конфигурации. JavaScript (6,2%) — целиком браузерное расширение (15 файлов, 5 653 строки). SQL (1,8%) — 7 файлов схем БД: *init*, *journal*, *ops*, *analytics*, *cdc_notify*, *age_schema*, *search_improvements*. HTML+CSS вместе (3,4%) — это web-UI чата на `src/web/` и `side panel/popup` расширения.

4.2. По топ-каталогам

Каталог	Файлов	Строк	Py-LOC
src/	349	58 050	52 659
scripts/	34	8 075	8 026
	24	4 789	2 597
features/	23	2 967	1 273
docs/	15	2 698	0
agents/	105	2 389	0
attic/	21	2 079	1 983
tests/	5	2 022	2 022
patches/	10	1 658	0
db/	7	1 603	0
extension/	10	1 586	0
mcp_servers/	38	1 369	0
harness/	37	992	82
config/	6	708	0
capabilities/	8	332	0
loops/	2	80	0
data/	0	0	0
deploy/	2	91	0
.github/	2	91	0

Таблица 4.2. Распределение по топ-каталогам (отсортировано по убыванию LOC)

Каталог `src/` содержит почти весь Python-код (52 659 LOC, 77% всего Python-кода проекта). Из оставшегося — `scripts/` хранит 34 скрипта (8 026 LOC, в основном тесты и миграции), `attic/` — 21 файл legacy-кода (1 983 LOC, помечен как архив), `tests/` — 5 pytest-модулей. Корневой каталог содержит 24 файла (4 789 LOC), из которых `setup.py` — 1 061 LOC, `first_run.py` — 680 LOC, `install.py` — 563 LOC. Каталог `data/` в финальной сборке пустой — это место для runtime-данных (SQLite/LanceDB), но профилей в нём нет (см. главу 3).

4.3. Подкаталоги `src/`

Подкаталог <code>src/</code>	Py-файлов	LOC	Назначение
<code>mcp_servers/</code>	88	15 596	37 MCP-серверов (один на подкаталог)
	20	7 481	<code>main.py</code> , <code>orchestrator</code> , <code>llm_client</code> , <code>llm_providers...</code>
<code>db/</code>	19	5 316	PostgreSQL pool, <code>vector_store</code> , AGE, CDC, backup

Подкаталог src/	Ру-файлов	LOC	Назначение
journal/	16	4 038	Event sourcing: recorder, replay, rollback
core/	20	3 535	Реестр: loader, schema, health, snapshot, history
memory/	25	3 445	5 слоёв памяти + retriever + embedders
supervisor/	7	2 087	Plan→Execute→Observe→Re-plan DAG
harness/	14	1 424	Тест-фреймворк: scenarios, fixtures, reporter
loop/	13	1 394	LoopController с reasoning/verification/retry
extraction/	13	1 073	PDF, DOCX, XLSX, Image, Audio extractors
onec_metadata/	6	809	Чтение метаданных 1С
background/	2	665	Фоновый task-runner с micro_tasks
onec_query/	6	646	Парсер запросов 1С
ollama/	4	630	Малая LLM: client, enricher, worker
prompts/	41	606	Шаблоны промптов для агентов

Таблица 4.3. Топ-15 подкаталогов src/ (по убыванию LOC)

4.4. Крупнейшие файлы

Файл	Строк	Роль
src/main.py	2 640	FastAPI-сервер: HTTP+WS, lifespan, админ-API
src/supervisor/supervisor.py	1 128	Supervisor: DAG-планы, Plan→Execute→Observe
setup.py	1 061	Единый скрипт настройки и запуска (9 шагов)
src/mcp_servers/code_analysis/server.py	1 002	MCP-сервер анализа кода (tree-sitter)
src/mcp_servers/filesystem/server.py	898	MCP-сервер файлов (29 инструментов)
src/mcp_servers/kubernetes/server.py	897	MCP-сервер Kubernetes (pods, deploys, helm)
src/memory/facade.py	778	5-слойная memory facade с dual-write PG
first_run.py	680	Мастер первого запуска
src/background/micro_tasks.py	662	Фоновые задачи с приоритетами
src/db/replicator.py	651	Репликатор журнала/сессий/планов в PG

Таблица 4.4. Топ-10 крупнейших Python-файлов по LOC

Файл `src/main.py` объёмом 2 640 строк — главная проблема `maintainability` проекта. Он содержит всю FastAPI-логику: объявление приложения, `lifespan` с инициализацией всех подсистем, HTTP-эндпоинты, WebSocket-обработчики чата, админ-API для управления агентами и MCP-серверами, а также кучу вспомогательных функций. Этот файл — кандидат на декомпозицию в FastAPI-роутеры (см. рекомендация 1 в главе 9). `supervisor.py` (1 128 строк) и `code_analysis/server.py` (1 002 строки) тоже великоваты, но в меньшей степени — у каждого чёткая ответственность.

Глава 5. Карта модулей

В этой главе — детальный разбор назначения каждого каталога в проекте. Для каждого указано количество Python-файлов и строк кода (где применимо), а также перечислены ключевые файлы внутри. Каталоги перечислены в порядке логической группировки, а не алфавитном — сначала ядро системы, затем декларативный слой, инфраструктура, тестирование и, наконец, операционные каталоги.

5.1. Ядро (`src/`)

Каталог	Py / LOC	Назначение
<code>src/</code>	20 / 7 481	<code>main.py</code> (2640 LOC, FastAPI), <code>orchestrator</code> , <code>llm_client</code> , <code>llm_providers</code> , <code>llm_errors</code> , <code>mcp_manager</code> , <code>policies</code> , <code>config</code> , <code>cli</code> , <code>web_chat</code> , <code>chat_import</code> , <code>bridge_store</code> , <code>web_cookies</code> , <code>route_analytics</code> , <code>cache</code> , <code>env_file</code> , <code>file_state</code> , <code>runtime_config</code> , <code>events</code>
<code>src/core/</code>	20 / 3 535	Реестр: <code>registry</code> , <code>loader</code> , <code>schema</code> , <code>health</code> , <code>snapshot</code> , <code>history</code> , <code>profiles</code> , <code>audit</code> , <code>rollback</code> , <code>metrics</code> , <code>file_watcher</code> , <code>background</code> , <code>migrations</code> , <code>runtime_config</code> , <code>features</code> — центральная координация всех подсистем
<code>src/loop/</code>	13 / 1 394	<code>LoopController</code> , <code>LoopContext</code> , <code>spec</code> , <code>state</code> , <code>steps</code> , <code>faults</code> , <code>telemetry</code> , <code>checks/</code> — декларативные циклы <code>reasoning/verification/retry</code> с бюджетами <code>max_iterations/max_tokens/max_duration_seconds</code>
<code>src/journal/</code>	16 / 4 038	Event sourcing: <code>recorder</code> (27 530 bytes — главный), <code>api</code> , <code>replay</code> , <code>rollback</code> , <code>retention</code> , <code>schema</code> , <code>store</code> , <code>integration</code> , <code>shadows</code> , <code>dependencies</code> , <code>mcp_server</code> , <code>cli</code> , <code>config</code> , <code>utils</code> , <code>reports</code> — журнал tool-calls с откатом
<code>src/supervisor/</code>	7 / 2 087	<code>supervisor</code> , <code>plans</code> , <code>dag</code> , <code>intents</code> , <code>session</code> , <code>models</code> — DAG-планы с циклом <code>Plan→Execute→Observe→Re-plan</code>
<code>src/agents/</code>	3 / 231	<code>AgentRuntime</code> — обёртки над <code>LoopController</code> (большая часть агентов декларативная, через YAML)
<code>src/background/</code>	2 / 665	Фоновый task-runner с <code>micro_tasks</code> (662 LOC)
<code>src/init/</code>	1 / ~80	Инициализационный подроутер
<code>src/capabilities/</code>	3 / 235	Реестр <code>capabilities</code> (<code>vision</code> , <code>ocr</code> , <code>whisper</code> , <code>media</code> , <code>storage</code> ...)
<code>src/storage/</code>	1 / ~80	Абстракция хранилища
<code>src/media/</code>	1 / ~80	Медиа-обработка <code>init</code>
<code>src/prompts/</code>	41 / 606	Шаблоны промптов для агентов (по одному на категорию)

Таблица 5.1. Каталоги ядра системы

5.2. Подсистемы данных

Каталог	Py / LOC	Назначение
src/db/	19 / 5 316	pool, vector_store (pgvector), memory_store, graph_store, hybrid_search (RRF), analytics, age_store (Cypher), age_sync, cdc (Kafka publisher), backup (pg_dump), pg_metrics (Prometheus), multi_instance, replicator, retention, search_helpers, autodetect, init_db
src/memory/	25 / 3 445	5 слоёв: working, episodic, semantic, vector, procedural + retriever, summarizer, embedders (bge-m3 → hashing fallback), facade (dual-write PG+SQLite), chunker (AST), graph (tree-sitter), tokens, user, store, config, graph_parsers/
src/extraction/	13 / 1 073	extractors/ (pdf, docx, xlsx, image, audio, text) — извлечение текста из файлов для индексации
src/cdc/	4 / ~400	Kafka publisher с буферизацией, notify_worker (PostgreSQL LISTEN/NOTIFY → Kafka)
src/cluster/	3 / ~300	Redis bus, heartbeat, WS bridge для multi-instance
src/ollama/	4 / 630	client, enricher, worker — фоновое обогащение памяти через малую LLM (qwen2.5:1.5b-instruct, ~1 ГБ VRAM)

Таблица 5.2. Подсистемы данных и памяти

5.3. MCP-серверы (src/mcp_servers/)

Каталог `src/mcp_servers/` содержит реализации 37 MCP-серверов (в финальной сборке; в `github-ready` — 39). Каждый сервер — это подкаталог с `__init__.py` и `server.py`, суммарно 88 файлов и 15 596 LOC Python. Серверы группируются по функциональным доменам:

Группа	Серверы	Назначение
Файлы и код	filesystem, code_analysis, lsp, debug, build*, data*	Доступ к файлам, статический анализ, LSP, отладка, сборка, ETL
БД	mysql, postgres, db_extended, migration	SQL-операции с MySQL/PostgreSQL, расширенные операции, миграции
1C	onec, onec_designer, onec_metadata, onec_query, onec_dcs, onec_forms, onec_tests	7 серверов для 1С:Предприятие — метаданные, запросы, дизайнер, формы, тесты
Git & CI	git, github, cicd	Git-операции, GitHub API, CI/CD pipelines
Browser	browser, frontend	Playwright browser MCP, фронтенд-инструменты
Инфра	kubernetes, network, monitoring, environment, storage	K8s pods/deployments/services, сеть, мониторинг, окружение, S3-like storage
Документы и медиа	document, image, media, http, security	DOCX, изображения (Vision), аудио/видео (Whisper), HTTP, security-сканер
Прочее	bridge, deepseek, memory, testing, journal, documentation	Bridge для расширения браузера, DeepSeek-провайдер, память, тесты, журнал, автодокументация

Таблица 5.3. 37 MCP-серверов по доменам (* — удалены в финальной сборке)

5.4. Декларативный слой и тестирование

Каталог	Структура	Назначение
agents/	35 подкаталогов × 3 файла = 105 файлов	Декларации агентов: agent.yaml + prompt.md + user.md. Поля: id, requires, mcp_servers, depends_on, priority, routing_hints, mode, dangerous_tools, prompt, runtime, ui
mcp_servers/	37 подкаталогов × 1 файл = 38 файлов	Декларации MCP-серверов: server.yaml с command, args, requires, tools[{name, danger}], process
capabilities/	8 YAML-файлов	Vision, OCR, Whisper, Media, Storage, Browser, 1c_metadata, 1c_query — операции и провайдеры
config/	6 YAML-файлов	settings, memory, models, alerting, extraction, litellm.example — конфигурация всех подсистем
loops/	2 YAML-файла	reasoning.yaml, verification.yaml — декларативные циклы для LoopController
features/	7 подкаталогов (journal, ops, bridge, cluster, history, memory, notes)	Каждая фича: feature.yaml + api.py + ui.js — унифицированный скелетон подсистемы
harness/	scenarios/ (9 категорий) + fixtures/ + tests/	Тест-фреймворк: smoke, dev, 1c, browser, image, document, media, storage, chaos scenarios
tests/	5 pytest-файлов	test_cookies_24a, test_journal, test_models_micro, test_pg_autodetect_24b, test_providers_24d — модульные тесты
scripts/	34 Python + 1 shell	Миграции (3), тесты (15+), build_extension, sign_firefox, e2e_db_pg, route_report, new_agent, new_feature, init_db, export_report, fake_provider, make_extension_icons
db/	7 SQL-файлов	init (база), journal (event sourcing), ops (операции), analytics (6 materialized views), cdc_notify (PostgreSQL→Kafka), age_schema (Cypher), search_improvements (Snowball+синонимы)
patches/	10 markdown-файлов + MANIFEST.md	Patch-манифесты MSG071-109: эволюция отдельных файлов (filesystem, code_analysis, web/style, main.py)
attic/	21 файл	Legacy: agents-legacy/, journal-pg/, mcp-journal-v1/ — архив для справки
deploy/	2 файла	env.example (шаблон переменных окружения), llm-agent.service (systemd unit)
docs/	14 markdown-файлов	ARCHITECTURE-V2, ARCHITECTURE, CAPABILITIES, FAQ, GETTING_STARTED, JOURNAL, DATABASE, providers, MCP_SERVERS, SECURITY, STRUCTURE, AGENTS, CONTRIBUTING, ROADMAP
.github/workflows/	2 YAML-файла	validate.yml (валидация деклараций на push/PR), harness.yml (прогон harness-сценариев)
extension/	panel/, popup/, shared/, icons/, dist/	Браузерное расширение LLM Agent Bridge для Chromium/Firefox — thin client, вся логика на локальном сервере

Таблица 5.4. Декларативный слой, тестирование и операционные каталоги

Глава 6. Зависимости и стек

Зависимости проекта описаны в трёх файлах: основной `requirements.txt` (~70 пакетов для обычного CPython 3.11+), усечённый `requirements-freethreaded.txt` (для free-threaded сборок 3.13t/3.14t без `lancedb`, `sentence-transformers`, `tree-sitter`, `duckdb`, `aiokafka`, `faster-whisper`), и дополнительные `requirements-psycopg3.txt` и `requirements-pg8000.txt` для переключения PG-драйвера. Установщик `install.py` автоматически определяет тип сборки Python (по суффиксу расширений `.cp314t-win_amd64.pyd`, флагу `Py_GIL_DISABLED`, `sys.abiflags` и пути интерпретатора — устойчивая детекция даже на Windows, где `sysconfig` не отдаёт `Py_GIL_DISABLED`) и берёт нужный файл требований. Если полный `requirements.txt` не сходится на Python 3.14+ (нет колёс либо `sdist`-конфликт `pydoc-markdown`), `pip`-установка автоматически повторяется на усечённом наборе.

6.1. Зависимости по категориям

Категория	Пакеты	Версии
Web-сервер	<code>fastapi</code> , <code>uvicorn[standard]</code> , <code>websockets</code> , <code>python-multipart</code>	0.115+, 0.30.6+, 13.1+, 0.0.9+
LLM и HTTP	<code>openai</code> , <code>httpx</code> , <code>httpx[http2]</code> , <code>aiohttp</code>	1.51+, 0.27.2+, 0.27.2+, 3.10+
MCP	<code>mcp</code> (pin <2.0 — критично! <code>mcp 2.x</code> удалил <code>@app.list_tools()</code>)	1.1.3+
Валидация	<code>pydantic</code> , <code>pydantic-settings</code> , <code>python-dotenv</code> , <code>PyYAML</code>	2.9.2+, 2.5.2+, 1.0.1+, 6.0.2+
PostgreSQL	<code>psycopg[binary,pool]</code> , <code>pgvector</code> , <code>psycopg_pool</code>	3.2+, 0.3+, 3.2+
MySQL/SQLite	<code>pymysql</code>	1.1.1+
Векторная память	<code>lancedb</code> , <code>pyarrow</code> , <code>sentence-transformers</code> (BAAI/bge-m3)	0.13+, 17.0+, 3.0+
Токенизация	<code>tiktoken</code> , <code>xxhash</code> , <code>sqlparse</code>	0.7+, 3.4+, 0.5+
Ollama (малая LLM)	<code>ollama</code> (qwen2.5:1.5b-instruct, ~1 ГБ VRAM)	0.4+
File-watching	<code>watchdog</code> , <code>psutil</code>	4.0+, 6.0+
Extraction	<code>pdfplumber</code> , <code>python-docx</code> , <code>openpyxl</code> , <code>Pillow</code> , <code>numpy</code> , <code>scikit-image</code> , <code>pytesseract</code>	0.11+, 1.1+, 3.1+, 10.4+, 2.0+, 0.24+, 0.3.13+
Tree-sitter (8 грамматик)	<code>tree-sitter</code> + <code>-python/-javascript/-typescript/-go/-rust/-java/-c-sharp/-php/-ruby</code>	0.23+ (все)
Browser MCP	<code>playwright</code> (install chromium отдельно)	1.48+
Media MCP	<code>faster-whisper</code> , <code>ffmpeg-python</code>	1.0+, 0.2+
Cluster	<code>redis</code>	5.0+
CDC + Kafka	<code>aiokafka</code>	0.11+
Dev tools (P0)	<code>pathspec</code> , <code>pgserver</code> (встроенный PG для тестов)	0.12+, 0.1.6+
Dev tools (P1)	<code>pip-audit</code> , <code>bandit</code> , <code>safety</code> , <code>PyGithub</code> , <code>alembic</code> , <code>pytest</code> , <code>pytest-cov</code> , <code>pytest-asyncio</code> , <code>respx</code> , <code>mutmut</code> , <code>py-spy</code> , <code>memray</code>	—

Категория	Пакеты	Версии
Dev tools (P2)	pymongo, elasticsearch, pymssql, griffe, pydoc-markdown, duckdb, pandas, cssutils	—
Kubernetes	kubernetes, kubernetes-asyncio	31.0+ (оба)
CLI	rich	13.9+

Таблица 6.1. Зависимости из requirements.txt по категориям

6.2. Особенности и риски стека

- (1) **Пиннинг mcp<2.0 — критичный defensive pin.** В комментарии к requirements.txt прямо указано: «mcp 2.x удалил декораторы @app.list_tools() — проект собран под 1.x». Это означает, что обновление mcp до 2.0 сломает все MCP-серверы. Рекомендуется добавить интеграционный тест против mcp 1.x, чтобы зафиксировать API-контракт (см. рекомендация 14 в главе 9).
- (2) **Все версии используют оператор >= без верхней границы,** кроме mcp. Это стандартный подход для активных проектов, но он несёт риск: любое обновление fastapi/pydantic/openai/psycorg может сломать сборку. Pin к compatible-release (~=) для критичных пакетов (fastapi, pydantic, psycorg, openai) обезопасит от сюрпризов (рекомендация 8).
- (3) **Dev-инструменты P1 (bandit, pip-audit, safety, pytest, mutmut) декларированы, но не enforced.** Нет pre-commit hooks, нет муру-gate в CI, нет обязательного покрытия тестами. Из 5 pytest-файлов в tests/ и ~30 test-скриптов в scripts/ — это тонкий слой регрессионного тестирования для системы такого размера (рекомендации 5, 6, 7).
- (4) **Free-threaded сборка Python 3.13t/3.14t — экспериментальная поддержка.** Усечённый набор requirements-freethreaded.txt исключает пакеты без t-колёс на PyPI (lancedb, sentence-transformers, tree-sitter, duckdb, aiokafka, faster-whisper); при их отсутствии система деградирует: векторное хранилище → postgres/файлы, embedder → hashing, граф кода/MCP data/CDC/whisper отключаются, PG-фичи возвращают 503 пока libpq не появится в PATH. Это продуманная стратегия graceful degradation, но она не покрыта интеграционными тестами — нужно добавить CI-матрицу на 3.13t.

Глава 7. Запуск и деплой

Проект поддерживает три способа запуска: локальный (через run.py), контейнерный (через docker compose — поднимает только инффра: PostgreSQL+pgvector, AGE, Kafka) и сервисный (через systemd unit в deploy/llm-agent.service). Основной путь для конечного пользователя — кросс-платформенный интерактивный установщик install.py (Windows/Linux/macOS) с флагом --auto для тихого режима в скриптах.

7.1. Точки входа и установщики

Файл	LOC	Роль
main.py	24	Shim на run.main() — чтобы «python main.py» из корня всегда работал

Файл	LOC	Роль
run.py	358	Точка входа: проверки Python/зависимостей → выбор PG-драйвера (psycopg3→psycopg2→pg8000) → автозапуск qwenпроху → uvicorn на 127.0.0.1:8000
install.py	563	Кросс-платформенный установщик: Python ≥3.11, .venv, pip, порт/PG/embedder, .env, схема БД, smoke-тест сервера
install.bat	399	Windows-версия установщика (использует install.py внутри)
install.sh	82	Linux/macOS-обёртка над install.py
setup.py	1061	Полный цикл: проверка Python+Node+PG+Ollama+Docker, pip, docker compose, ollama, .env, схема, миграции, health, uvicorn
first_run.py	680	Мастер первого запуска: PROJECT_ROOT, data/ каталоги, SQLite/LanceDB, AI-настройки (DeepSeek-куки или OpenAI-API)
run.bat	139	Windows-обёртка над run.py
run.sh	26	Linux/macOS-обёртка над run.py
build_exe.bat	44	Сборка одиночного exe через PyInstaller (Windows only, 5-15 минут)

Таблица 7.1. Точки входа и установочные скрипты

7.2. Docker Compose — инфраструктура

Файл docker-compose.yml описывает четыре контейнера. Важно: **основное приложение не контейнеризовано** — Docker используется только для инфраструктуры (PostgreSQL, AGE, Kafka). Сам llm-agent запускается нативно через python run.py. Это упрощает отладку, но усложняет деплой (рекомендация 11 — добавить Dockerfile для основного приложения).

Контейнер	Образ	Порт	Назначение
postgres	pgvector/pgvector:pg16	5432	Основной PostgreSQL с pgvector 0.7+ для векторной памяти. Схема применяется автоматически через docker-entrypoint-initdb.d (init.sql, journal.sql, ops.sql, analytics.sql, search_improvements.sql, cdc_notify.sql)
age-postgres	apache/age:PG16_latest	5433	Apache AGE — отдельная база для property-graph (Cypher-запросы). Схема age_schema.sql применяется при первом старте
kafka	apache/kafka:3.8.0	9092	Kafka в KRaft-режиме (без ZooKeeper). 6 topics для CDC-событий. auto_create_topics_enabled=true
kafka-ui	provectuslabs/kafka-ui:latest	8080	Web-UI для Kafka (опционально, depends_on: kafka healthy)

Таблица 7.2. Docker Compose сервисы

7.3. Systemd и nginx

Для Linux-деплойа как сервиса в deploy/llm-agent.service лежит systemd unit: Type=simple, User=llmagent, WorkingDirectory=/opt/llm-agent, ExecStart=/opt/llm-agent/.venv/bin/python run.py --port 8000, Restart=on-failure с RestartSec=5, After=network.target postgresql.service. Файл nginx.conf описывает обратный прокси с sticky sessions через ip_hash на два инстанса (instance-a:8001,

instance-b:8002) — это_confirms multi-instance поддержку, реализованную через Redis bus и LISTEN/NOTIFY в коде. Лимит open files поднят до 8192 (LimitNOFILE=8192) — обоснованно для системы с потоками WebSocket и MCP stdio-серверами.

7.4. Браузерное расширение

Расширение LLM Agent Bridge (Chromium/Firefox) собирается скриптом scripts/build_extension.py → extension/dist/bridge-{chromium,firefox}/ + готовые ZIP. Для постоянной установки в Firefox (без режима отладки) — скрипт scripts/sign_firefox.py с АМО JWT-ключами: пересобирает раскладку, гоняет web-ext lint, подписывает channel unlisted (свое распространение) или listed (публикация на АМО с ревью). Первая подпись навсегда привязывает id llm-agent-bridge@local к аккаунту АМО; повторная подпись той же версии запрещена — нужно поднять version в манифестах. Расширение реализовано как thin client: вся логика на локальном сервере, расширение только захватывает страницы/выделения и пересылает на сервер через WS. Контекстное меню («Захватить выделенное в LLM Agent», «Спросить агента про выделенное», «Суммаризировать страницу»), горячая клавиша Alt+Shift+B, омнибокс `ag` + Tab.

Глава 8. Качество кода и безопасность

Сканирование выполнено скриптом scripts/analyze_security.py, который обходит все 408 Python-файлов проекта (исключая .git/ и extension/dist/) и ищет 16 паттернов: eval(), exec(), shell=True в subprocess, os.system(), pickle.loads(), небезопасный yaml.load(), hardcoded passwords, assert в проде, print(), bare except, TODO/FIXME/HACK/XXX, noqa, type:ignore. Результаты — в таблице 8.1. Общее впечатление: **гигиена кода хорошая** — нет ни одного критичного security-дефекта (eval, shell=True, pickle, yaml.load без safe).

Паттерн	Hits	Оценка	Комментарий
eval()	0	✓ Отлично	Критичных eval-вызовов нет вообще
shell=True (subprocess)	0	✓ Отлично	Все subprocess используют shell=False (безопасно)
pickle.loads()	0	✓ Отлично	Десериализация pickle не используется (нет риска RCE)
yaml.load() (unsafe)	0	✓ Отлично	Используется только yaml.safe_load()
bare except	0	✓ Отлично	Все except либо типизированы, либо # noqa: BLE001 в мастерах
os.system()	1	⚠ Приемлемо	install.py:53 — os.system("") для пробуждения ANSI-цветов на Windows, команда пустая
exec()	4	⚠ Легитимно	Все в src/mcp_servers/debug/server.py — debug-MCP исполняет пользовательские скрипты через cProfile.run (legitimate use case)
assert (в проде)	4	⚠ Риск	llm_client.py:184, lsp/client.py:111/160 — assert исчезает с -O, заменить на raise

Паттерн	Hits	Оценка	Комментарий
hardcoded_password	2	✓ Тесты	Оба в scripts/test_bridge.py с fake credentials (k-args/s-args)
TODO	7	✓ Не критично	5 из 7 — это DEFAULT STRING LITERALS в code_analysis (MCP ищет TODO!), не реальные TODO
FIXME	3	✓ Не критично	Все 3 — строковые литералы в code_analysis MCP
HACK / XXX	4	✓ Не критично	Все — литералы в code_analysis и в onec_query/parser.py (метаданные Справочник.XXX)
print()	452	⚠ Долг	Большинство в install.py/setup.py/first_run.py (CLI-вывод легитимен), но часть в проде — вынести в logging
# noqa	106	⚠ Долг	BLE001 в мастерах (осознанно), E402 для поздних импортов — легитимно
# type: ignore	11	⚠ Долг	Большинство в scripts/test_*.py — не блокирующе, но без туру-gate типизация декоративна

Таблица 8.1. Результаты сканирования 408 Python-файлов на 16 паттернов

8.1. Подробности по рискам

exec() в debug-MCP — легитимная функция. Все 4 вызова находятся в src/mcp_servers/debug/server.py в контексте cProfile.run('exec(open(script).read())') — это MCP-сервер для отладки пользовательских Python-скриптов с профилированием. Агент debug имеет mode: destructive и dangerous_tools — все операции требуют явного подтверждения пользователя через approval-gate. Риск минимальный, но стоит добавить песочницу (Docker/Pyodide) — рекомендация 13.

4 assert-выражения в проде. Конкретно: src/llm_client.py:184 (assert last_exc is not None — invariant после retry-цикла), src/mcp_servers/lsp/client.py:111 (assert self.process — проверка запущенного LSP-сервера) и lsp/client.py:160 (assert self.process and self.process.stdin). Python с флагом -O (или -OO) удаляет assert-выражения, что превратит эти invariant-checks в молчаливые пор. Заменить на явный if not ...: raise RuntimeError(...) (рекомендация 5). Четвёртый assert — в harness/tests/test_scenarios.py:33, это тестовый код, нормально.

452 print() — основной технический долг. Подавляющее большинство — в CLI-скриптах (install.py/setup.py/first_run.py), где print() в консоль легитимен. Но часть попадает в серверный код: src/main.py, src/mcp_manager.py, src/llm_providers.py и др. — там должны быть logging.info() / logger.warning() с уровнем и контекстом (рекомендация 12).

8.2. Архитектурные долги

Сам docs/ARCHITECTURE-V2.md прямо признаёт несколько архитектурных долгов: (1) «src/agents/*_agent.py (11 файлов): большинство не используется нигде, кроме src/agents/__init__.py; система живёт на BaseAgent + декларациях» — legacy-классы, подлежащие переносу в attic/ (рекомендация 9); (2) «on_step мёртв. Callback объявлен в LoopContext, но контроллер его не вызывает — UI не видит шагов» — нужно восстановить вызов; (3) «Импорт-гигиена. src/journal/__init__.py тянет

storage → db.pool → psycorg: импорт хука без установленного psycorg валился с traceback» — теперь кэшируется и молчит, но сам факт — сигнал о правиле; (4) «UI-вкладки монолитны. src/web/app.js — единый файл; journal.js сделан самодостаточным как исключение, а не как правило» — нужно унифицировать feature-скелетон (рекомендация 13).

Глава 9. Выводы и рекомендации

На основе анализа двух архивов сформулированы 14 практических рекомендаций, отсортированных по приоритету (P0 — критичные, P1 — важные, P2 — желательные). Каждая рекомендация содержит проблему, конкретное действие и оценку трудозатрат.

#	Приоритет	Рекомендация	Трудозатраты
1	P0	Декомпозировать src/main.py (2 640 LOC) в FastAPI-роутеры: chat.py, agents.py, mcp.py, admin.py, journal.py, ops.py. Сейчас один файл содержит HTTP+WS+lifespan+админ-API — главный риск maintainability	2-3 спринта
2	P0	Восстановить data/profiles/*.json в финальной сборке (4 файла: default, development, production, lc_development). Код ожидает их найти; без них первый запуск не найдёт встроенных пресетов	< 1 часа
3	P0	Документировать разницу архивов в README: «github-ready — снимок репозитория с git-историей и dev-MCP; final — чистый релиз без экспериментов»	1 час
4	P1	Добавить pre-commit hooks для bandit/pip-audit/ruff. Сейчас инструменты декларированы в requirements.txt (P1), но не enforced — нет pre-commit-config.yaml	1 спринт
5	P1	Заменить assert в проде на явные исключения. Конкретно: src/llm_client.py:184, src/mcp_servers/lsp/client.py:111/160. assert исчезает с -O	1 час
6	P1	Добавить mypy-gate в CI с --strict для src/. Сейчас 11 # type: ignore и нет mypy — типизация декоративна	1 спринт
7	P1	Добавить unit-test workflow в .github/workflows/ (сейчас только validate.yml для деклараций и harness.yml для сценариев, нет pytest). Покрытие ключевых модулей: orchestrator, llm_client, journal/recorder, memory/facade	1-2 спринта
8	P1	Закрепить версии критичных пакетов: fastapi, pydantic, psycorg, openai, mcp — сменить >= на ~= (compatible release). Защита от сюрпризов при обновлениях	2 часа
9	P2	Перенести src/agents/*_agent.py (11 legacy-файлов) в attic/agents-legacy/. Сам ARCHITECTURE-V2.md признаёт: «большинство не используется нигде, кроме __init__.py»	1 час
10	P2	Контейнеризовать основное приложение (Dockerfile). Сейчас в docker-compose только инфра (PG, AGE, Kafka), а llm-agent запускается нативно — усложняет деплой	1 спринт
11	P2	Добавить .github/SECURITY.md с responsible disclosure процессом. Проект работает с секретами (DeepSeek-куки, API-ключи, PG-credentials) — нужен публичный канал репорта	1 час

#	Приоритет	Рекомендация	Трудозатраты
12	P2	Вынести <code>print()</code> из серверного кода в <code>logging</code> . Оставить <code>print()</code> только в CLI-скриптах (<code>install.py/setup.py/first_run.py</code>). 452 <code>print()</code> — основной техдолг	1 спринт
13	P2	Унифицировать <code>feature</code> -скелетон: применить паттерн <code>features/*/feature.yaml + api.py + ui.js</code> (как в <code>journal</code> , <code>ops</code> , <code>bridge</code>) ко всем подсистемам. Сейчас <code>journal</code> сделан «как исключение, а не как правило»	2 спринта
14	P2	Добавить интеграционный тест против <code>mcp 1.x API</code> (зафиксировать <code>@app.list_tools()</code> контракт). <code>mcp 2.x</code> удалил декораторы — пиннинг <code><2.0</code> есть, но тест отсутствует	1 спринт

Таблица 9.1. 14 практических рекомендаций по приоритетам (P0/P1/P2)

9.1. Дорожная карта развития

Спринт 1 (P0): восстановить профили (рекомендация 2) и документировать разницу архивов (рекомендация 3) — это быстрые победы, выполняются за день. Параллельно начать декомпозицию `src/main.py` (рекомендация 1) — самый объёмный рефакторинг, 2-3 спринта, но критичный для дальнейшей `developability` системы.

Спринты 2-3 (P1): `quality gate` — `pre-commit` с `bandit/ruff` (4), муру в CI (6), замена `assert` (5), `pin` версий (8). Параллельно — расширение тестового покрытия (7): добавить `pytest workflow` в `.github/workflows/`, написать тесты на `orchestrator`, `llm_client`, `journal/recorder`, `memory/facade`. Эти работы блокируют дальнейшее развитие — без `quality gate` любое новое изменение рискует регрессию.

Спринты 4-5 (P2): зрелость проекта. Контейнеризация основного приложения (10), `unified feature skeleton` (13), `mcp 1.x` интеграционный тест (14), `cleanup legacy` (9), `SECURITY.md` (11), `logging` вместо `print` (12). После этих работ проект будет готов к `production-deploy` с поддержкой `multi-instance`, `autoscaling` и `observable` через `Prometheus + OpenTelemetry`.

9.2. Заключительная оценка

Проект **llm-agent** — зрелая мультиагентная платформа с продуманной декларативной архитектурой, качественной гигиеной кода (ноль критичных `security`-дефектов) и подробной документацией (14 `markdown`-файлов в `docs/`). Два архива показывают эволюцию: «`github-ready`» — это снимок репозитория с историей и экспериментальными модулями (`build`, `data`), «`final`» — чистый релиз без `dev`-экспериментов. Удаление `runtime`-профилей из `final`, скорее всего, непреднамеренная утечка, требующая исправления. Главные направления развития: декомпозиция `src/main.py` (2 640 строк), добавление `quality gate` (`pre-commit`, муру, CI-тесты) и контейнеризация основного приложения. С учётом 14 рекомендаций проект готов к выходу из `alpha`-стадии в `production-deploy`.